

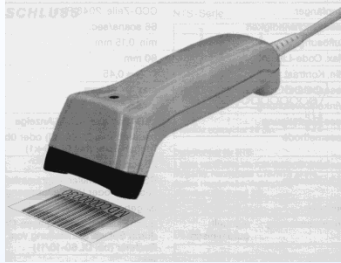
Data Mining

Lecture # 8

Mining Frequent Patterns
Associations and Correlations

Introduction

Motivation



{butter, bread, milk, sugar}

{butter, flour, milk, sugar}

{butter, eggs, milk, salt}

{eggs}

{butter, flour, milk, salt, sugar}

DB of Sales Transactions

Market basket analysis

- Which products are frequently purchased together?
- Applications
 - Improvement of store layouts
 - Cross marketing
 - Attached mailings/add-on sales

Market-Basket Data

- A large set of **items**, e.g., things sold in a supermarket.
- A large set of **baskets**, each of which is a small set of the items, e.g., the things one customer buys on one day.

Market-Baskets – (2)

- Really, a general many-to-many mapping (association) between two kinds of things, where the one (the **baskets**) is a set of the other (the **items**)
 - But we ask about connections among “items,” not “baskets.”
- The technology focuses on **common events**, not rare events (“long tail”).

Frequent Itemsets

- Given a set of transactions, find combinations of items (**itemsets**) that occur frequently

Market-Basket transactions

Items: {Bread, Milk, Diaper, Beer, Eggs, Coke}

TID	Items
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

Support $s(I)$: number of transactions that contain itemset I

Examples of frequent itemsets $s(I) \geq 3$

{Bread}: 4
{Milk}: 4
{Diaper}: 4
{Beer}: 3
{Diaper, Beer}: 3
{Milk, Bread}: 3

Applications – (1)

- **Items** = products; **baskets** = sets of products someone bought in one trip to the store.
- **Example application**: given that many people buy beer and diapers together:
 - Run a sale on diapers; raise price of beer.
- Only useful if many buy diapers & beer.

Applications – (2)

- Baskets = Web pages; items = words.
- Example application: Unusual words appearing together in a large number of documents, e.g., “Heer” and “Ranjha”, may indicate an interesting relationship.

Transaction data: a set of documents

- **A text document data set. Each document is treated as a “bag” of keywords**

doc1: Student, Teach, School

doc2: Student, School

doc3: Teach, School, City, Game

doc4: Baseball, Basketball

doc5: Basketball, Player, Spectator

doc6: Baseball, Coach, Game, Team

doc7: Basketball, Team, City, Game

Applications – (3)

- **Baskets** = sentences; **items** = documents containing those sentences.
- **Example application:** Items that appear together too often could represent plagiarism.
- Notice items do not have to be “in” baskets.

Definition: Frequent Itemset

□ Itemset

- A collection of one or more items
 - Example: {Milk, Bread, Diaper}

□ k-itemset

- An itemset that contains k items

□ Support (σ)

- **Count:** Frequency of occurrence of an itemset
- E.g. $\sigma(\{\text{Milk, Bread, Diaper}\}) = 2$
- **Fraction:** Fraction of transactions that contain an itemset
- E.g. $s(\{\text{Milk, Bread, Diaper}\}) = 40\%$

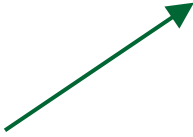
□ Frequent Itemset

- An itemset whose support is greater than or equal to a *minsup* threshold

TID	Items
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

$$s(I) \geq \text{minsup}$$

An example



t1:	Beef, Chicken, Milk
t2:	Beef, Cheese
t3:	Cheese, Boots
t4:	Beef, Chicken, Cheese
t5:	Beef, Chicken, Clothes, Cheese, Milk
t6:	Chicken, Clothes, Milk
t7:	Chicken, Milk, Clothes

- Transaction data

- Assume:

minsup = 30%

minconf = 80%

- An example **frequent itemset**:

{Chicken, Clothes, Milk} [sup = 3/7]

- **Association rules** from the itemset:

Clothes → Milk, Chicken [sup = 3/7, conf = 3/3]

...

...

Clothes, Chicken → Milk, [sup = 3/7, conf = 3/3]

Some more definitions

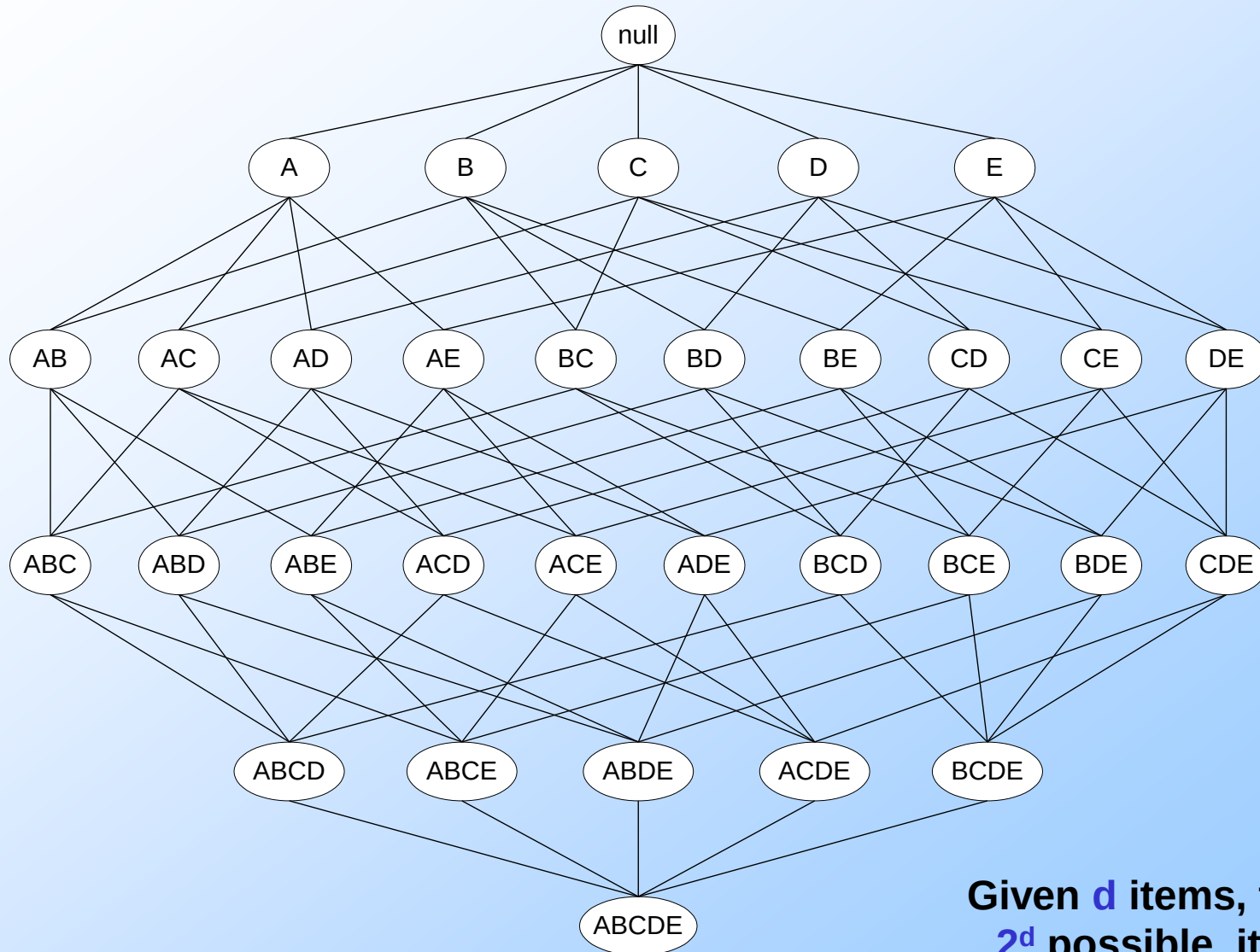
- **Close Itemset** : An itemset X is closed in a data set D if there exists no proper super-itemset Y such that Y has the same support count as X in D .
- **closed frequent itemset**: An itemset X is a **closed frequent itemset** in set D if X is both closed and frequent in D .
- **Maximal frequent itemset** : An itemset X is a **maximal frequent itemset** (or **max-itemset**) in a data set D if X is frequent, and there exists no super-itemset Y such that $X \subset Y$ and Y is frequent in D .

Mining Frequent Itemsets task

- **Input:** A set of transactions T , over a set of items I
- **Output:** All itemsets with items in I having
 - $\text{support} \geq \text{minsup}$ threshold
- Problem parameters:
 - $N = |T|$: number of transactions
 - $d = |I|$: number of (distinct) items
 - w : max width of a transaction
 - Number of possible itemsets?
- Scale of the problem:
 - Walmart sells 100,000 items and can store billions of baskets.
 - The Web has billions of words and many billions of pages.

$$M = 2^d$$

The itemset lattice



Given d items, there are 2^d possible itemsets

Initial Definition of Association Rules (ARs) Mining

- Association rules define relationship of the form:

$$\mathbf{A \rightarrow B}$$

- Read as A implies B, where A and B are sets of binary valued attributes represented in a data set.
- Association Rule Mining (ARM) is then the process of finding all the ARs in a given DB.

Association Rule: Basic Concepts

- Given: (1) database of transactions, (2) each transaction is a list of items (purchased by a customer in a visit)
- Find: all rules that correlate the presence of one set of items with that of another set of items
 - E.g., *98% of students who study Databases and C++ also study Algorithms*
- Applications
 - *Home Electronics* $\Rightarrow$ * (What other products should the store stocks up?)
 - *Attached mailing* in direct marketing
 - *Web page navigation* in Search Engines (first *page a* $\rightarrow$ *page b*)
 - *Text mining* if IT companies $\rightarrow$ Microsoft

Some Notation

D = A data set comprising n records and **m** binary valued attributes.

I = The set of m attributes, $\{i_1, i_2, \dots, i_m\}$, represented in D .

Itemset = Some subset of I . Each record in D is an itemset.

In depth Definition of ARs Mining

- Association rules define relationship of the form:

$$\mathbf{A} \rightarrow \mathbf{B}$$

- Read as **A** implies **B**
- Such that **$A \subset I$** , **$B \subset I$** , **$A \cap B = \emptyset$** (A and B are *disjoint*) and **$A \cup B \subseteq I$** .
- In other words an AR is made up of an itemset of cardinality 2 or more.

ARM Problem Definition (1)

Given a database D we wish to find (*Mine*) all the itemsets of cardinality 2 or more, contained in D, and then use these item sets to create association rules of the form $A \rightarrow B$.

The number of potential itemsets of cardinality 2 or more is:

$$2^m - m - 1$$

If $m=5$, #potential itemsets = 26

If $m=20$, #potential itemsets = 1048556

So now we do not want to find “all the itemsets of cardinality 2 or more, contained in D”, we only want to find the **interesting** itemsets of cardinality 2 or more, contained in D.

Association Rules Measurement

The most commonly used “interestingness” measures are:

1. Support
2. Confidence

Itemset Support

- **Support**: A measure of the frequency with which an itemset occurs in a DB.

$$\text{supp}(\mathbf{A}) = \frac{\# \text{ records that contain } \mathbf{A}}{m}$$

- If an itemset has support higher than some specified threshold we say that the itemset is *supported* or *frequent* (some authors use the term *large*).
- Support threshold is normally set reasonably low (say) 1%.

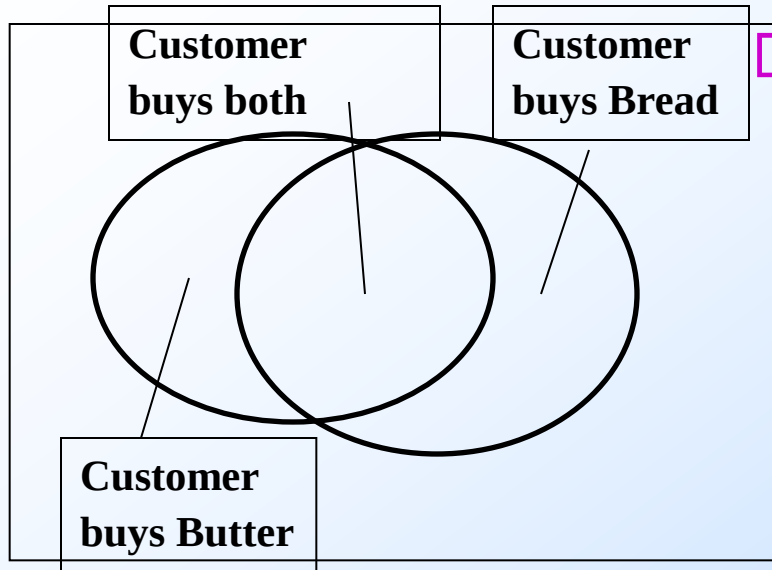
Confidence

- **Confidence**: A measure, expressed as a ratio, of the support for an AR compared to the support of its antecedent.

$$\text{conf}(A \rightarrow B) = \frac{\text{supp}(A \cup B)}{\text{supp}(A)}$$

- We say that we are confident in a rule if its confidence exceeds some threshold (normally set reasonably high, say, 80%).

Rule Measures: Support and Confidence



Find all the rules $X \& Y \Rightarrow Z$ with minimum confidence and support

- support, s , probability that a transaction contains $\{X \& Y \& Z\}$
- confidence, c , conditional probability that a transaction having $\{X \& Y\}$ also contains Z

Transaction ID	Items Bought
2000	A,B,C
1000	A,C
4000	A,D
5000	B,E,F

Let minimum support 50%, and minimum confidence 50%, we have

- $A \Rightarrow C$ (50%, 66.6%)
- $C \Rightarrow A$ (50%, 100%)

ARM Problem Definition (2)

- Given a database D we wish to find all the frequent itemsets (F) and then use this knowledge to produce high confidence association rules.
- Note: Finding F is the most computationally expensive part, once we have the frequent sets generating ARs is straight forward

BRUTE FORCE

a	6	cd	3	abce	0
b	6	acd	1	de	3
ab	3	bcd	1	ade	1
c	6	abcd	0	bde	1
ac	3	e	6	abde	0
bc	3	ae	3	cde	1
abc	1	be	3	acde	0
d	6	abe	1	bcde	0
ad	6	ce	3	abcde	0
bd	3	ace	1		
abd	1	bce	1		

List all possible combinations in an array.

For each record:

1. Find all combinations.
2. For each combination index into array and increment support by 1.

Then generate rules

Support threshold = 5%

(count of 1.5) where
total # of trans = 30

a	6	cd	3	abce	0
b	6	acd	1	de	3
ab	3	bcd	1	ade	1
c	6	abcd	0	bde	1
ac	3	e	6	abde	0
bc	3	ae	3	cde	1
abc	1	be	3	acde	0
d	6	abe	1	bcde	0
ad	6	ce	3	abcde	0
bd	3	ace	1		
abd	1	bce	1		

Frequent Sets (F):

ab (3) ac (3) bc (3)

ad (6) bd (3) cd (3)

ae (3) be (3) ce (3)

de (3)

Rules:

From ab we can
develop two rules
as

$a \rightarrow b$ $\text{conf} = 3/6 = 50\%$

$b \rightarrow a$ $\text{conf} = 3/6 = 50\%$

Etc.

BRUTE FORCE

Advantages:

- 1) Very efficient for data sets with small numbers of attributes (<20).

Disadvantages:

- 1) Given 20 attributes, number of combinations is $2^{20}-1 = 1048576$. Therefore array storage requirements will be 4.2MB.
- 2) Given a data sets with (say) 100 attributes it is likely that many combinations will not be present in the data set --- therefore store only those combinations present in the dataset!

Mining Association Rules—An Example

Transaction ID	Items Bought
2000	A,B,C
1000	A,C
4000	A,D
5000	B,E,F

Min. support 50%
Min. confidence 50%

Frequent Itemset	Support
{A}	75%
{B}	50%
{C}	50%
{A,C}	50%

For rule $A \Rightarrow C$:

support = support($\{A \cup C\}$) = 50%

confidence = support($\{A \cup C\}$)/support($\{A\}$) =
66.6%

The **Apriori** principle:

Any subset of a frequent itemset must be frequent

Joining the itemsets

The join step: To find L_k , a set of **candidate** k -itemsets is generated by joining L_{k-1} with itself. This set of candidates is denoted C_k . Let l_1 and l_2 be itemsets in L_{k-1} . The notation $l_i[j]$ refers to the j th item in l_i (e.g., $l_1[k-2]$ refers to the second to the last item in l_1). For efficient implementation, Apriori assumes that items within a transaction or itemset are sorted in lexicographic order. For the $(k-1)$ -itemset, l_i , this means that the items are sorted such that $l_i[1] < l_i[2] < \dots < l_i[k-1]$. The join, $L_{k-1} \bowtie L_{k-1}$, is performed, where members of L_{k-1} are joinable if their first $(k-2)$ items are in common. That is, members l_1 and l_2 of L_{k-1} are joined if $(l_1[1] = l_2[1]) \wedge (l_1[2] = l_2[2]) \wedge \dots \wedge (l_1[k-2] = l_2[k-2]) \wedge (l_1[k-1] < l_2[k-1])$. The condition $l_1[k-1] < l_2[k-1]$ simply ensures that no duplicates are generated. The resulting itemset formed by joining l_1 and l_2 is $\{l_1[1], l_1[2], \dots, l_1[k-2], l_1[k-1], l_2[k-1]\}$.

Mining Frequent Itemsets: the Key Step

- Find the *frequent itemsets*: the sets of items that have minimum support
 - A subset of a frequent itemset must also be a frequent itemset
 - i.e., if $\{AB\}$ is a frequent itemset, both $\{A\}$ and $\{B\}$ should be a frequent itemset
 - Iteratively find frequent itemsets with cardinality from 1 to k (k -itemset)
- Use the frequent itemsets to generate association rules.

Subset testing

- Any $(k - 1)$ -itemset that is not frequent cannot be a subset of a frequent k -itemset.
- Hence, if any $(k - 1)$ -subset of a candidate k -itemset is not in L_{k-1} , then the candidate cannot be frequent either and so can be removed from C_k .
- This **subset testing** can be done quickly by maintaining a hash tree of all frequent itemsets.

The Apriori Algorithm — Example

Database D

TID	Items
100	1 3 4
200	2 3 5
300	1 2 3 5
400	2 5

Scan D

C_1

itemset	sup.
{1}	2
{2}	3
{3}	3
{4}	1
{5}	3

L_1

itemset	sup.
{1}	2
{2}	3
{3}	3
{5}	3

L_2

itemset	sup
{1 3}	2
{2 3}	2
{2 5}	3
{3 5}	2

C_2

itemset	sup
{1 2}	1
{1 3}	2
{1 5}	1
{2 3}	2
{2 5}	3
{3 5}	2

Scan D

C_2

itemset
{1 2}
{1 3}
{1 5}
{2 3}
{2 5}
{3 5}

C_3

itemset
{2 3 5}

Scan D

L_3

itemset	sup
{2 3 5}	2

The Apriori Algorithm

□ Pseudo-code:

C_k : Candidate itemset of size k

L_k : frequent itemset of size k

$L_1 = \{\text{frequent items}\};$

for ($k = 1; L_k \neq \emptyset; k++$) **do begin**

C_{k+1} = candidates generated from L_k ;

for each transaction t in database **do**

increment the count of all candidates in C_{k+1}

that are contained in t

L_{k+1} = candidates in C_{k+1} with min_support

end

return $\cup_k L_k$;

Important Details of Apriori

◆ How to generate candidates?

- ◆ Step 1: self-joining L_k
- ◆ Step 2: pruning

◆ How to count supports of candidates?

◆ Example of Candidate-generation

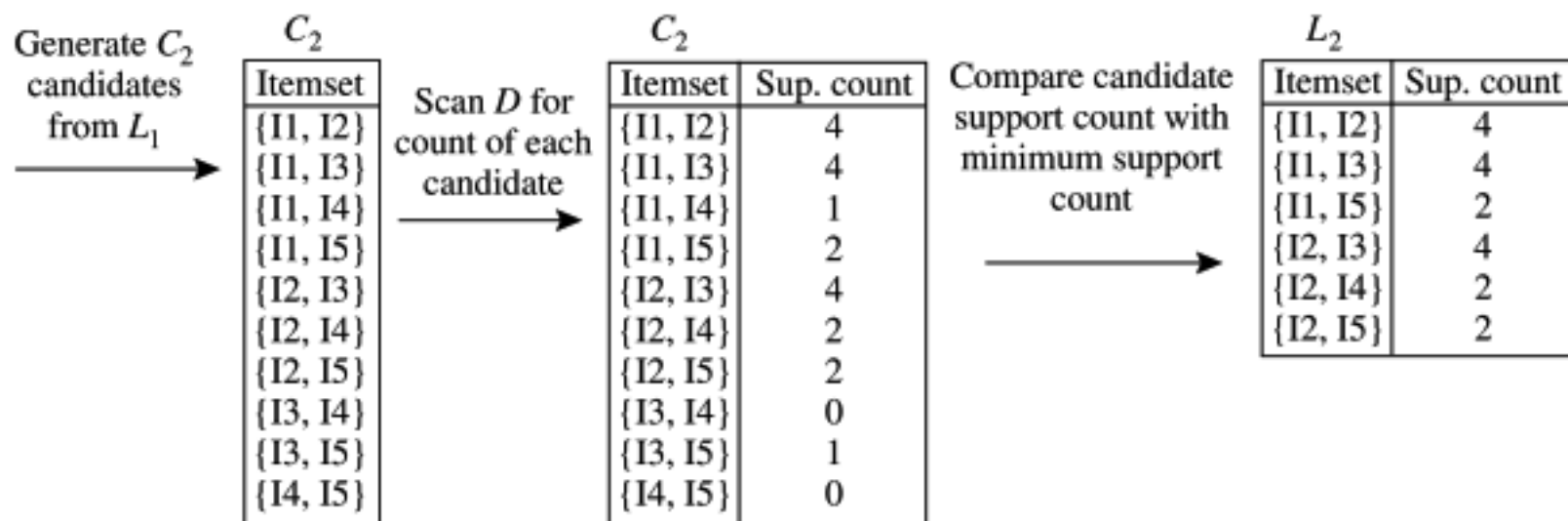
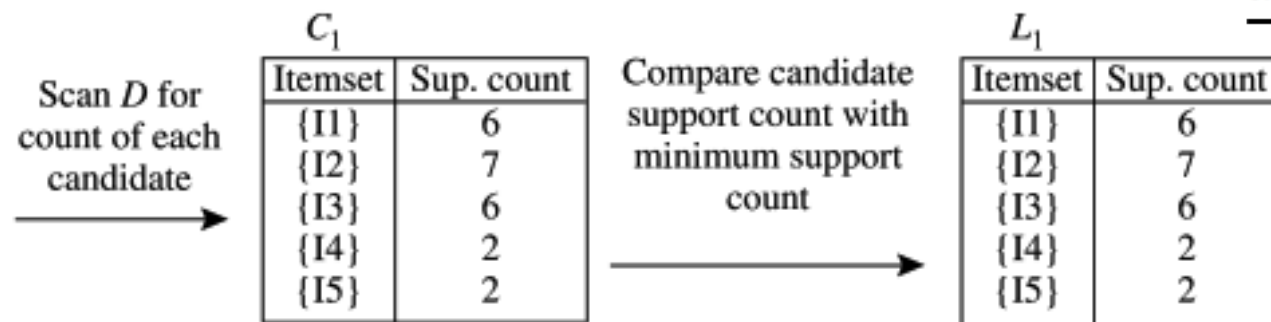
- ◆ $L_3 = \{abc, abd, acd, ace, bcd\}$
- ◆ Self-joining: $L_3 \bowtie L_3$
 - $abcd$ from abc and abd
 - $acde$ from acd and ace
- ◆ Pruning:
 - $acde$ is removed because ade is not in L_3
- ◆ $C_4 = \{abcd\}$

Table 6.1 Transactional Data for an *AllElectronics* Branch

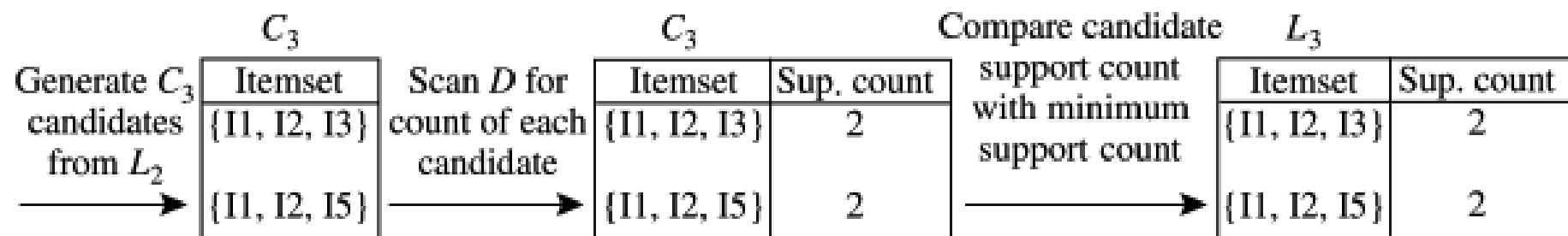
<i>TID</i>	<i>List of item_IDs</i>
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

Table 6.1 Transactional Data for an *Allelectronics* Branch

<i>TID</i>	<i>List of item.IDs</i>
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3



Join: $C_3 = L_2 \bowtie L_2 = \{\{I1, I2\}, \{I1, I3\}, \{I1, I5\}, \{I2, I3\}, \{I2, I4\}, \{I2, I5\}\}$
 $\bowtie \{\{I1, I2\}, \{I1, I3\}, \{I1, I5\}, \{I2, I3\}, \{I2, I4\}, \{I2, I5\}\}$
 $= \{\{I1, I2, I3\}, \{I1, I2, I5\}, \{I1, I3, I5\}, \{I2, I3, I4\}, \{I2, I3, I5\}, \{I2, I4, I5\}\}.$



Itemset	Sup. count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

Itemset	Sup. count
{I1, I2}	4
{I1, I3}	4
{I1, I5}	2
{I2, I3}	4
{I2, I4}	2
{I2, I5}	2

Table 6.1 Transactional Data for an *AllElectronics* Branch

TID	List of item IDs
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

Example 6.4 Generating association rules. Let's try an example based on the transactional data for *AllElectronics* shown before in Table 6.1. The data contain frequent itemset $X = \{I1, I2, I5\}$. What are the association rules that can be generated from X ? The nonempty subsets of X are $\{I1, I2\}$, $\{I1, I5\}$, $\{I2, I5\}$, $\{I1\}$, $\{I2\}$, and $\{I5\}$. The resulting association rules are as shown below, each listed with its confidence:

$\{I1, I2\} \Rightarrow I5, \text{ confidence} = 2/4 = 50\%$
 $\{I1, I5\} \Rightarrow I2, \text{ confidence} = 2/2 = 100\%$
 $\{I2, I5\} \Rightarrow I1, \text{ confidence} = 2/2 = 100\%$
 $I1 \Rightarrow \{I2, I5\}, \text{ confidence} = 2/6 = 33\%$
 $I2 \Rightarrow \{I1, I5\}, \text{ confidence} = 2/7 = 29\%$
 $I5 \Rightarrow \{I1, I2\}, \text{ confidence} = 2/2 = 100\%$

For $A \rightarrow C$

support = support($\{A \cup C\}$)

confidence = support($\{A \cup C\}$) / support($\{A\}$)

If the minimum confidence threshold is, say, 70%, then only the second, third, and last rules are output, because these are the only ones generated that are strong. Note that, unlike conventional classification rules, association rules can contain more than one conjunct in the right side of the rule. ■